```
void allocateResources() {
    sortOutpostsByPriority();
    vector<bool> fulfilled(outposts.size(), false);
    for (Outpost &outpost : outposts){
      if (outpost.weight_needed <= 0)
        continue;
      UAV *selected_uav = nullptr;
      double min_total_time = numeric_limits<double>::max();
      for (UAV &uav : uavs){
        double distance_to_outpost = calculateDistance(uav.current_position,
outpost.coordinates);
        double distance_back_to_base = calculateDistance(outpost.coordinates, coordinates);
        double total_distance = distance_to_outpost + distance_back_to_base;
        if (uav.canFulfill(outpost.weight_needed, distance_to_outpost, outpost.deadline -
uav.total_time))
        {
          double potential_time = uav.total_time + uav.calculateTime(total_distance);
          if (potential_time < min_total_time) {
            min_total_time = potential_time;
            selected_uav = &uav;
          }
        }
      }
      if (selected_uav) {
        double distance = calculateDistance(selected_uav->current_position,
outpost.coordinates);
        double delivery_weight = min(outpost.weight_needed, selected_uav-
>available_capacity);
        selected_uav->deliver(distance, delivery_weight, outpost.coordinates);
        cout << fixed << setprecision(2);
        cout << "UAV " << selected_uav->id << " delivered " << delivery_weight
          << " units to Outpost " << outpost.id << " (Distance: " << distance
```

```
            << " units, Time: " << selected_uav->calculateTime(distance) << " units)." << endl;
          outpost.weight_needed -= delivery_weight;
          if (outpost.weight_needed <= 0){
            fulfilled[outpost.id] = true;
            cout << "Outpost " << outpost.id << " fulfilled." << endl;
          }
          selected_uav->resetToBase(coordinates);
          cout << "UAV " << selected_uav->id << " returned to base for refuel/reassignment." <<
endl;
          cout << "   Remaining Capacity: " << selected_uav->available_capacity << " units" <<
endl;
          cout << "   Total Fuel Consumed: " << selected_uav->fuel_consumed << " units" << endl;
          cout << "   Total Time Used: " << selected_uav->total_time << " units" << endl;
        }
        else
        {
          cout << "No UAV can fulfill Outpost " << outpost.id << " within the deadline." << endl;
        }
      }
    }
```